Guia Completo de MySQL: do Básico ao Avançado

Este guia cobre o MySQL 8.x, do primeiro `SELECT` à otimização de consultas. Cada seção traz explicação curta, código pronto para executar e as pegadinhas específicas do MySQL.

> **Como usar:** execute os exemplos no seu servidor, na ordem das seções. Todos usam a mesma base de exemplo (uma loja), criada na seção 2.

Nível	Seções	Ao final você será capaz de
Básico	1 a 3	Criar bancos e tabelas, inserir e consultar dados
Intermediário	4 e 5	Agrupar, juntar tabelas e escrever subconsultas e CTEs
Avançado	6 a 9	Usar window functions, otimizar com índices, controlar transações e programar no banco
Profissional	10 e 11	Proteger, fazer backup e praticar com exercícios

1. Introdução e Ambiente

O **MySQL** é um SGBD relacional de código aberto, mantido pela Oracle, muito usado em aplicações web (WordPress, WooCommerce, Laravel, sistemas PHP e Node). Ele usa SQL como linguagem e organiza dados em **bancos (schemas)**, que contêm **tabelas**.

Grupos de comandos SQL

Grupo	Função	Comandos
DDL	Define a estrutura	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `RENAME`
DML	Altera dados	`INSERT`, `UPDATE`, `DELETE`, `REPLACE`
DQL	Consulta dados	`SELECT`
DCL	Controla permissões	`GRANT`, `REVOKE`
TCL	Controla transações	`START TRANSACTION`, `COMMIT`, `ROLLBACK`, `SAVEPOINT`

Engines de armazenamento

O MySQL permite escolher como cada tabela é gravada. Na prática, use sempre **InnoDB**.

Engine	Transações	Chaves estrangeiras	Lock	Quando usar
InnoDB (padrão)	Sim	Sim	Por linha	Praticamente tudo
MyISAM	Não	Não	Por tabela	Legado; evite em projetos novos
MEMORY	Não	Não	Por tabela	Dados temporários em RAM

Instalação e conexão

```
# Ubuntu/Debian
sudo apt update && sudo apt install mysql-server
sudo mysql_secure_installation

# Conectar pelo terminal
mysql -u root -p
mysql -h 127.0.0.1 -P 3306 -u usuario -p nome_do_banco
```

Para uma interface gráfica, use **MySQL Workbench**, **DBeaver** ou **phpMyAdmin**. Outra opção rápida é subir com Docker:

```
docker run --name mysql8 -e MYSQL_ROOT_PASSWORD=senha123 -p 3306:3306 -d
mysql:8
```

Comandos úteis do cliente

```
SHOW DATABASES;            -- lista os bancos
USE loja;                  -- seleciona o banco
SHOW TABLES;               -- lista as tabelas
DESCRIBE clientes;         -- mostra colunas (ou DESC clientes)
SHOW CREATE TABLE clientes;-- mostra o DDL completo
SELECT VERSION();          -- versão do servidor
```

> **Dica:** terminar comandos com `;` é obrigatório no cliente de terminal. Comentários: `-- texto` (com espaço após os dois hífens), `# texto` e `/* bloco */`.

2. Tipos de Dados, Base de Exemplo e DDL

Tipos de dados do MySQL

Categoria	Tipos	Observações
Inteiros	`TINYINT`, `SMALLINT`, `INT`, `BIGINT`	`UNSIGNED` dobra o limite positivo; `TINYINT(1)` é usado como booleano
Decimais exatos	`DECIMAL(p,s)`	Use para dinheiro, ex.: `DECIMAL(10,2)`
Decimais aproximados	`FLOAT`, `DOUBLE`	Nunca para valores financeiros
Texto curto	`CHAR(n)`, `VARCHAR(n)`	`VARCHAR` até 65.535 bytes por linha
Texto longo	`TEXT`, `MEDIUMTEXT`, `LONGTEXT`	Não aceitam `DEFAULT` literal
Data e hora	`DATE`, `TIME`, `DATETIME`, `TIMESTAMP`, `YEAR`	`TIMESTAMP` converte por fuso e vai até 2038
Lógico	`BOOLEAN`	Apelido de `TINYINT(1)`
Listas	`ENUM('a','b')`, `SET`	Úteis, mas dificultam mudanças futuras
Outros	`JSON`, `BLOB`, `BINARY(16)`	`BINARY(16)` é comum para UUID

> **Importante:** use sempre o charset **utf8mb4** (e não `utf8`, que no MySQL guarda só 3 bytes por caractere e quebra emojis). Prefira a collation `utf8mb4_0900_ai_ci` no MySQL 8.

Criando o banco

```
CREATE DATABASE loja
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_0900_ai_ci;

USE loja;
```

Base de exemplo (usada no guia inteiro)

```
CREATE TABLE clientes (
  id        INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nome      VARCHAR(100) NOT NULL,
  email     VARCHAR(120) NOT NULL UNIQUE,
  cidade    VARCHAR(60),
  ativo     BOOLEAN NOT NULL DEFAULT TRUE,
  criado_em DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE produtos (
  id        INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nome      VARCHAR(100) NOT NULL,
  categoria VARCHAR(50),
  preco     DECIMAL(10,2) NOT NULL CHECK (preco >= 0)
) ENGINE=InnoDB;

CREATE TABLE pedidos (
  id         INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  cliente_id INT UNSIGNED NOT NULL,
  data       DATE NOT NULL,
  status     ENUM('aberto','pago','cancelado') NOT NULL DEFAULT 'aberto',
  CONSTRAINT fk_pedido_cliente FOREIGN KEY (cliente_id)
    REFERENCES clientes(id) ON DELETE RESTRICT
) ENGINE=InnoDB;

CREATE TABLE itens_pedido (
  pedido_id  INT UNSIGNED NOT NULL,
  produto_id INT UNSIGNED NOT NULL,
  quantidade INT UNSIGNED NOT NULL,
  PRIMARY KEY (pedido_id, produto_id),
  FOREIGN KEY (pedido_id)  REFERENCES pedidos(id),
  FOREIGN KEY (produto_id) REFERENCES produtos(id)
) ENGINE=InnoDB;
```

`AUTO_INCREMENT` gera o id sozinho. Depois de um `INSERT`, `LAST_INSERT_ID()` devolve o id criado.

Constraints

Constraint	Função
`PRIMARY KEY`	Identifica a linha; único e não nulo
`FOREIGN KEY`	Garante que o valor existe na tabela referenciada

Constraint	Função
`UNIQUE`	Impede valores repetidos (aceita vários `NULL`)
`NOT NULL`	Proíbe `NULL`
`DEFAULT`	Valor padrão quando não informado
`CHECK`	Valida uma regra (aplicada de fato a partir do MySQL 8.0.16)

Ações de chave estrangeira: `ON DELETE CASCADE`, `SET NULL`, `RESTRICT` (padrão) e `NO ACTION`.

ALTER, DROP e TRUNCATE

```
ALTER TABLE clientes ADD COLUMN telefone VARCHAR(20) AFTER email;
ALTER TABLE clientes MODIFY COLUMN telefone VARCHAR(30) NOT NULL;
ALTER TABLE clientes CHANGE COLUMN cidade municipio VARCHAR(60);  -- renomeia
ALTER TABLE clientes DROP COLUMN telefone;
ALTER TABLE clientes ADD INDEX idx_cidade (municipio);
RENAME TABLE clientes TO clientes_antigo;

TRUNCATE TABLE itens_pedido;   -- apaga tudo, zera AUTO_INCREMENT, não volta atrás
DROP TABLE IF EXISTS itens_pedido;
```

> **Atenção:** `DROP` e `TRUNCATE` são comandos DDL e fazem *commit implícito*: não dá para dar `ROLLBACK` neles.

3. DML e SELECT

INSERT, UPDATE e DELETE

```
INSERT INTO clientes (nome, email, cidade) VALUES
  ('Ana',   'ana@email.com',   'Rio de Janeiro'),
  ('Bruno', 'bruno@email.com', 'Niterói'),
  ('Carla', 'carla@email.com', NULL);

UPDATE clientes SET cidade = 'São Paulo' WHERE id = 2;

DELETE FROM clientes WHERE id = 3;
```

Variações próprias do MySQL:

Comando	O que faz
`INSERT IGNORE`	Ignora linhas que violam `UNIQUE` ou PK, sem erro
`INSERT ... ON DUPLICATE KEY UPDATE`	Insere ou atualiza se a chave já existe (upsert)
`REPLACE INTO`	Apaga a linha antiga e insere a nova (cuidado com FKs)
`INSERT INTO ... SELECT`	Insere o resultado de uma consulta

```
INSERT INTO clientes (id, nome, email)
VALUES (1, 'Ana Souza', 'ana@email.com')
ON DUPLICATE KEY UPDATE nome = VALUES(nome);
```

> **Regra de ouro:** `UPDATE` e `DELETE` sem `WHERE` afetam a tabela inteira. Teste primeiro com um `SELECT` usando o mesmo `WHERE`. O modo `sql_safe_updates` (padrão do Workbench) bloqueia esse tipo de acidente.

SELECT: estrutura

```
SELECT   colunas
FROM     tabela
WHERE    filtro_de_linhas
GROUP BY agrupamento
HAVING   filtro_de_grupos
ORDER BY ordenacao
LIMIT    quantidade OFFSET deslocamento;
```

Ordem **lógica** de execução: `FROM/JOIN`, `WHERE`, `GROUP BY`, `HAVING`, `SELECT`, `DISTINCT`, `ORDER BY`, `LIMIT`. Por isso o `WHERE` não enxerga aliases do `SELECT`, mas `ORDER BY` e `HAVING` enxergam no MySQL.

```
SELECT * FROM clientes;                      -- evite * em produção
SELECT nome, email FROM clientes;
SELECT DISTINCT cidade FROM clientes;
SELECT nome, preco * 0.9 AS preco_com_desconto FROM produtos;
```

Filtros no WHERE

Operador	Exemplo	Observação
`=`, `<>`, `!=`, `<`, `>`, `<=`, `>=`	`preco >= 100`	
`AND`, `OR`, `NOT`	`a AND (b OR c)`	`AND` vence `OR`; use parênteses
`IN`	`cidade IN ('Niterói','São Paulo')`	
`BETWEEN`	`preco BETWEEN 50 AND 200`	Inclui os dois extremos
`LIKE`	`nome LIKE 'A%'`	`%` = qualquer sequência, `_` = um caractere
`REGEXP`	`nome REGEXP '^(Ana|Bia)'`	Expressões regulares
`IS NULL` / `IS NOT NULL`	`cidade IS NULL`	Nunca use `= NULL`
`<=>`	`cidade <=> NULL`	Igualdade que trata `NULL` (exclusivo do MySQL)

Por padrão, comparações de texto no MySQL **não diferenciam maiúsculas de minúsculas** (collations `_ci`). Para forçar, use `WHERE BINARY nome = 'Ana'`.

NULL

`NULL` significa "desconhecido". Qualquer comparação com `NULL` resulta em `NULL`, nunca em verdadeiro.

```
SELECT NULL = NULL;                       -- NULL
SELECT IFNULL(cidade, 'Não informada') FROM clientes;
SELECT COALESCE(telefone, email, 'sem contato') FROM clientes;
SELECT 10 / NULLIF(quantidade, 0) FROM itens_pedido;   -- evita divisão por zero
```

Ordenação e paginação

```
SELECT * FROM produtos ORDER BY preco DESC, nome ASC;
SELECT * FROM produtos ORDER BY preco DESC LIMIT 10;
SELECT * FROM produtos ORDER BY id LIMIT 10 OFFSET 20;   -- página 3 de 10 em
10
SELECT * FROM produtos ORDER BY id LIMIT 20, 10;         -- forma curta:
LIMIT offset, qtd
```

> **Desempenho:** `OFFSET` grande é lento, pois o MySQL lê e descarta as linhas anteriores. Para listas longas, pagine por chave: `WHERE id > :ultimo_id ORDER BY id LIMIT 10`.

4. Funções e Agregações

Funções de texto

Função	Exemplo	Resultado
`CONCAT`	`CONCAT('Ana', ' ', 'Lima')`	`Ana Lima`
`CONCAT_WS`	`CONCAT_WS('-', 2026, 10, 8)`	`2026-10-8`
`UPPER` / `LOWER`	`UPPER('rio')`	`RIO`
`LENGTH` / `CHAR_LENGTH`	`CHAR_LENGTH('ação')`	`4` (`LENGTH` conta bytes: 6)
`SUBSTRING`	`SUBSTRING('banana', 2, 3)`	`ana`
`LEFT` / `RIGHT`	`LEFT('banana', 3)`	`ban`
`TRIM`	`TRIM(' oi ')`	`oi`
`REPLACE`	`REPLACE('a-b', '-', '/')`	`a/b`
`LOCATE`	`LOCATE('@', email)`	posição do `@`
`LPAD`	`LPAD(7, 3, '0')`	`007`

> **Diferença do padrão:** no MySQL, `||` é o operador `OR`, não concatenação. Use sempre `CONCAT()`.

Funções numéricas

```
SELECT ROUND(19.987, 2), CEIL(4.1), FLOOR(4.9), ABS(-5), MOD(10, 3), POWER(2, 10), RAND();
SELECT 5 / 2, 5 DIV 2, 5 % 2;    -- 2.5000, 2, 1  (DIV = divisão inteira)
```

Funções de data e hora

Função	Para que serve
`NOW()`, `CURDATE()`, `CURTIME()`	Data e hora atuais
`DATE_ADD(data, INTERVAL 7 DAY)`	Somar intervalo (também `DATE_SUB`)
`DATEDIFF(d1, d2)`	Dias entre duas datas
`TIMESTAMPDIFF(MONTH, d1, d2)`	Diferença em outra unidade
`YEAR()`, `MONTH()`, `DAY()`, `DAYOFWEEK()`	Extrair partes
`DATE_FORMAT(data, '%d/%m/%Y')`	Formatar
`STR_TO_DATE('08/10/2026', '%d/%m/%Y')`	Converter texto em data
`LAST_DAY(data)`	Último dia do mês

```
SELECT DATE_FORMAT(NOW(), '%d/%m/%Y %H:%i');          -- 08/10/2026 14:30
SELECT * FROM pedidos WHERE data >= CURDATE() - INTERVAL 30 DAY;
```

Formatos mais usados: `%Y` ano (4 dígitos), `%m` mês, `%d` dia, `%H` hora (24h), `%i` minutos, `%s` segundos, `%M` nome do mês.

Condicionais: IF, CASE

```
SELECT nome, IF(preco > 100, 'Caro', 'Barato') AS faixa FROM produtos;

SELECT nome,
  CASE
    WHEN preco < 50  THEN 'Barato'
    WHEN preco < 200 THEN 'Médio'
    ELSE 'Caro'
  END AS faixa
FROM produtos;
```

Funções de agregação

Função	Resultado
`COUNT(*)`	Total de linhas
`COUNT(coluna)`	Linhas com valor não nulo
`COUNT(DISTINCT coluna)`	Valores distintos
`SUM`, `AVG`, `MIN`, `MAX`	Soma, média, menor e maior (ignoram `NULL`)
`GROUP_CONCAT(col SEPARATOR ', ')`	Junta valores de um grupo em um texto
`JSON_ARRAYAGG`, `JSON_OBJECTAGG`	Agrega em JSON

GROUP BY e HAVING

```
SELECT categoria,
       COUNT(*)            AS qtd,
       ROUND(AVG(preco),2) AS preco_medio,
       GROUP_CONCAT(nome ORDER BY nome SEPARATOR ', ') AS produtos
FROM produtos
GROUP BY categoria
HAVING COUNT(*) > 2
ORDER BY qtd DESC;
```

- `WHERE` filtra linhas **antes** de agrupar.
- `HAVING` filtra grupos **depois** de agrupar e pode usar agregações.
- Colunas no `SELECT` fora de agregações devem estar no `GROUP BY`. O modo `ONLY_FULL_GROUP_BY` (padrão do MySQL 8) recusa consultas que violam isso.

`WITH ROLLUP` adiciona linhas de subtotal e total geral:

```
SELECT categoria, SUM(preco) AS total
FROM produtos
GROUP BY categoria WITH ROLLUP;   -- a última linha, com categoria NULL, é o
total geral
```

Pivotando com CASE

```
SELECT
  SUM(status = 'pago')      AS pagos,       -- no MySQL, comparação vira 1 ou
0
  SUM(status = 'cancelado') AS cancelados,
  SUM(status = 'aberto')    AS abertos
FROM pedidos;
```

5. JOINs, Subconsultas e CTEs

Tipos de JOIN

Tipo	Resultado	No MySQL
`INNER JOIN`	Só linhas com par nas duas tabelas	Suportado
`LEFT JOIN`	Todas da esquerda e os pares da direita (`NULL` sem par)	Suportado
`RIGHT JOIN`	Todas da direita e os pares da esquerda	Suportado
`FULL OUTER JOIN`	Todas de ambas	**Não existe**: emule com `UNION`
`CROSS JOIN`	Todas as combinações possíveis	Suportado
Self join	Tabela ligada a ela mesma	Use aliases diferentes

```
-- Pedidos com o nome do cliente
SELECT p.id, c.nome, p.data, p.status
FROM pedidos p
INNER JOIN clientes c ON c.id = p.cliente_id;

-- Clientes que nunca fizeram pedido (anti-join)
SELECT c.nome
FROM clientes c
LEFT JOIN pedidos p ON p.cliente_id = c.id
WHERE p.id IS NULL;

-- Total gasto por cliente (4 tabelas)
SELECT c.nome, SUM(i.quantidade * pr.preco) AS total
FROM clientes c
JOIN pedidos p      ON p.cliente_id = c.id
JOIN itens_pedido i ON i.pedido_id = p.id
```

```
JOIN produtos pr    ON pr.id = i.produto_id
WHERE p.status = 'pago'
GROUP BY c.id, c.nome
ORDER BY total DESC;
```

Emulando FULL OUTER JOIN:

```
SELECT * FROM a LEFT JOIN b ON a.id = b.a_id
UNION
SELECT * FROM a RIGHT JOIN b ON a.id = b.a_id;
```

> **Armadilha clássica:** em um `LEFT JOIN`, um filtro sobre a tabela da direita no `WHERE` elimina as linhas `NULL` e transforma o resultado em `INNER JOIN`. Coloque essa condição no `ON`.

Outras formas úteis: `JOIN ... USING (coluna)` quando as colunas têm o mesmo nome nas duas tabelas, e `UPDATE` / `DELETE` com `JOIN`:

```
UPDATE produtos pr
JOIN itens_pedido i ON i.produto_id = pr.id
SET pr.preco = pr.preco * 1.05
WHERE i.quantidade > 100;
```

Subconsultas

```
-- Escalar: produtos acima da média
SELECT nome, preco FROM produtos
WHERE preco > (SELECT AVG(preco) FROM produtos);

-- IN: clientes com pedido pago
SELECT nome FROM clientes
WHERE id IN (SELECT cliente_id FROM pedidos WHERE status = 'pago');

-- EXISTS: checa existência (costuma ser mais eficiente)
SELECT nome FROM clientes c
WHERE EXISTS (SELECT 1 FROM pedidos p WHERE p.cliente_id = c.id);

-- Tabela derivada: o alias é obrigatório no MySQL
SELECT t.categoria, t.media
FROM (SELECT categoria, AVG(preco) AS media FROM produtos GROUP BY categoria)
AS t
WHERE t.media > 100;
```

> **Cuidado com `NOT IN`:** se a subconsulta devolver qualquer `NULL`, o resultado fica vazio. Prefira `NOT EXISTS` ou um `LEFT JOIN ... IS NULL`.

Uma subconsulta **correlacionada** (que usa colunas da consulta externa) roda uma vez por linha externa e pode ser lenta; avalie trocar por `JOIN`.

CTEs (`WITH`, MySQL 8+)

Dão nome a uma consulta intermediária e deixam o código legível.

```
WITH vendas AS (
  SELECT p.cliente_id, SUM(i.quantidade * pr.preco) AS total
  FROM pedidos p
  JOIN itens_pedido i ON i.pedido_id = p.id
  JOIN produtos pr    ON pr.id = i.produto_id
  GROUP BY p.cliente_id
)
SELECT c.nome, v.total
FROM vendas v
JOIN clientes c ON c.id = v.cliente_id
WHERE v.total > 1000;
```

CTE recursiva, para hierarquias como organograma ou categorias em árvore:

```
WITH RECURSIVE arvore AS (
  SELECT id, nome, gerente_id, 1 AS nivel
  FROM funcionarios WHERE gerente_id IS NULL
  UNION ALL
  SELECT f.id, f.nome, f.gerente_id, a.nivel + 1
  FROM funcionarios f
  JOIN arvore a ON f.gerente_id = a.id
)
SELECT * FROM arvore;
```

UNION, INTERSECT e EXCEPT

```
SELECT cidade FROM clientes
UNION            -- junta e remove duplicados
SELECT cidade FROM fornecedores;
-- UNION ALL mantém duplicados e é mais rápido
-- INTERSECT e EXCEPT existem a partir do MySQL 8.0.31
```

6. Window Functions e JSON

Window functions (MySQL 8+)

Calculam valores sobre um conjunto de linhas **sem colapsar** o resultado, ao contrário do `GROUP BY`. A forma geral é `função() OVER (PARTITION BY ... ORDER BY ...)`.

Função	Para que serve
`ROW_NUMBER()`	Numera as linhas: 1, 2, 3, 4
`RANK()`	Ranking com empates que pulam posições: 1, 2, 2, 4
`DENSE_RANK()`	Ranking com empates sem pular: 1, 2, 2, 3
`NTILE(n)`	Divide em n faixas iguais (quartis, decis)
`LAG(col, n)` / `LEAD(col, n)`	Valor da linha anterior ou seguinte
`FIRST_VALUE` / `LAST_VALUE`	Primeiro ou último valor da janela
`SUM/AVG/COUNT ... OVER`	Acumulados e médias móveis

```
-- Ranking de produtos por preço dentro de cada categoria
SELECT categoria, nome, preco,
  ROW_NUMBER() OVER (PARTITION BY categoria ORDER BY preco DESC) AS posicao,
  RANK()       OVER (PARTITION BY categoria ORDER BY preco DESC) AS
rank_preco
FROM produtos;

-- Total acumulado e variação em relação ao mês anterior
SELECT mes, total,
  SUM(total) OVER (ORDER BY mes) AS acumulado,
  total - LAG(total) OVER (ORDER BY mes) AS variacao
FROM vendas_mensais;

-- Média móvel de 3 períodos
SELECT data, valor,
  AVG(valor) OVER (ORDER BY data ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS
media_movel
FROM vendas_diarias;

-- Participação de cada produto no total
SELECT nome, preco, ROUND(preco * 100 / SUM(preco) OVER (), 1) AS pct
FROM produtos;
```

Padrão clássico, o "top N por grupo" (3 produtos mais caros de cada categoria). Como o MySQL não permite filtrar window function no `WHERE`, use uma CTE:

```
WITH ranking AS (
  SELECT categoria, nome, preco,
    ROW_NUMBER() OVER (PARTITION BY categoria ORDER BY preco DESC) AS rn
  FROM produtos
)
SELECT * FROM ranking WHERE rn <= 3;
```

Janelas nomeadas evitam repetição: `SELECT ... SUM(x) OVER w ... WINDOW w AS (PARTITION BY a ORDER BY b)`.

JSON no MySQL

O tipo `JSON` valida o conteúdo, armazena em formato binário e permite extrair valores com a sintaxe de caminho.

```
CREATE TABLE eventos (
  id    INT AUTO_INCREMENT PRIMARY KEY,
  dados JSON NOT NULL
);

INSERT INTO eventos (dados)
VALUES ('{"tipo": "login", "usuario": {"id": 7, "nome": "Ana"}, "tags":
["web", "mobile"]}');

SELECT dados->>'$.usuario.nome'            AS usuario,   -- ->> devolve texto
sem aspas
       dados->'$.tags[0]'                  AS primeira_tag,
       JSON_EXTRACT(dados, '$.usuario.id') AS id_usuario
FROM eventos
WHERE dados->>'$.tipo' = 'login'
  AND JSON_CONTAINS(dados->'$.tags', '"web"');

UPDATE eventos SET dados = JSON_SET(dados, '$.visto', TRUE) WHERE id = 1;
```

Função	Uso
`JSON_EXTRACT`, `->`, `->>`	Ler valores
`JSON_SET`, `JSON_INSERT`, `JSON_REPLACE`	Alterar valores
`JSON_REMOVE`	Remover chaves
`JSON_ARRAYAGG`, `JSON_OBJECTAGG`	Montar JSON a partir de linhas

Função	Uso
`JSON_TABLE`	Transformar JSON em linhas e colunas

> **Quando usar:** JSON serve para atributos variáveis e dados semiestruturados. Para dados que você filtra e junta o tempo todo, prefira colunas normais, que são mais rápidas e indexáveis (para indexar um campo JSON, crie uma *generated column* e indexe-a).

7. Índices, EXPLAIN e Otimização

Como os índices funcionam

Um índice é uma estrutura ordenada (no InnoDB, uma **B-tree**) que permite achar linhas sem varrer a tabela inteira. O custo: ocupam espaço e deixam `INSERT`, `UPDATE` e `DELETE` um pouco mais lentos.

Tipo	Como criar	Uso
Primário	`PRIMARY KEY (id)`	No InnoDB, define a ordem física da tabela (índice clusterizado)
Único	`UNIQUE INDEX idx (email)`	Busca rápida e garantia de unicidade
Comum	`INDEX idx (cidade)`	Filtros, `JOIN`, `ORDER BY`
Composto	`INDEX idx (cliente_id, data)`	Consultas que filtram por várias colunas
Prefixo	`INDEX idx (nome(10))`	Colunas de texto longas
`FULLTEXT`	`FULLTEXT INDEX idx (titulo, corpo)`	Busca textual com `MATCH ... AGAINST`
Funcional (8.0.13+)	`INDEX idx ((LOWER(email)))`	Indexar o resultado de uma expressão

```
CREATE INDEX idx_pedidos_cliente_data ON pedidos (cliente_id, data);
SHOW INDEX FROM pedidos;
DROP INDEX idx_pedidos_cliente_data ON pedidos;
```

Regra do índice composto (leftmost prefix)

Um índice em `(a, b, c)` serve para filtros em `a`, em `a, b` e em `a, b, c`, mas **não** para filtros só em `b` ou só em `c`. Ponha primeiro as colunas comparadas com `=` e, depois, a coluna de intervalo ou de ordenação.

O que impede o uso do índice

Padrão ruim	Alternativa
`WHERE YEAR(data) = 2026`	`WHERE data >= '2026-01-01' AND data < '2027-01-01'`
`WHERE nome LIKE '%silva'`	`LIKE 'silva%'` ou índice `FULLTEXT`
`WHERE telefone = 21999990000` (coluna texto, valor numérico)	`WHERE telefone = '21999990000'`
`WHERE preco + 10 > 100`	`WHERE preco > 90`
`OR` entre colunas diferentes sem índice em ambas	`UNION` de duas consultas

EXPLAIN: lendo o plano de execução

```
EXPLAIN SELECT * FROM pedidos WHERE cliente_id = 10 ORDER BY data;
EXPLAIN ANALYZE SELECT ...;   -- MySQL 8.0.18+: executa e mostra tempos reais
```

Coluna	O que olhar
`type`	Do melhor ao pior: `const`, `eq_ref`, `ref`, `range`, `index`, **`ALL`** (varredura completa)
`key`	Índice realmente usado (`NULL` = nenhum)
`rows`	Estimativa de linhas lidas; quanto menor, melhor
`Extra`	`Using index` é ótimo; `Using filesort` e `Using temporary` merecem atenção

> **Roteiro de otimização:** (1) meça com `EXPLAIN`; (2) confira se `type` é `ALL` em tabela grande; (3) crie o índice para o `WHERE` / `JOIN` / `ORDER BY`; (4) rode `EXPLAIN` de novo e compare.

Boas práticas de desempenho

- Liste só as colunas necessárias em vez de `SELECT *`.
- Indexe colunas de chave estrangeira e as usadas em `JOIN`.
- Prefira `EXISTS` a `IN` com subconsultas grandes e `UNION ALL` a `UNION` quando duplicados não importam.
- Insira em lote: `INSERT ... VALUES (...), (...), (...)` é muito mais rápido que um `INSERT` por linha.
- Evite `OFFSET` grande; pagine por chave.
- Use `ANALYZE TABLE` para atualizar estatísticas e `OPTIMIZE TABLE` para recuperar espaço após muitos `DELETE`.
- Ative o **slow query log** para descobrir consultas lentas:

```
SET GLOBAL slow_query_log = 'ON';
SET GLOBAL long_query_time = 1;   -- registra consultas acima de 1 segundo
SHOW FULL PROCESSLIST;            -- consultas em execução agora
```

8. Transações, ACID e Locks

Uma **transação** agrupa vários comandos em uma unidade: ou todos valem, ou nenhum. Só funciona em tabelas **InnoDB**.

```
START TRANSACTION;
  UPDATE contas SET saldo = saldo - 100 WHERE id = 1;
  UPDATE contas SET saldo = saldo + 100 WHERE id = 2;
COMMIT;        -- confirma tudo
-- ROLLBACK;   -- desfaz tudo

SAVEPOINT antes_do_desconto;
ROLLBACK TO SAVEPOINT antes_do_desconto;   -- desfaz só até o ponto marcado
```

Por padrão o MySQL roda em **autocommit**: cada comando vira uma transação própria e é confirmado na hora. `START TRANSACTION` suspende isso até o `COMMIT` ou `ROLLBACK`.

ACID

Letra	Propriedade	Significado
A	Atomicidade	Tudo ou nada
C	Consistência	O banco passa de um estado válido para outro válido

Letra	Propriedade	Significado
I	Isolamento	Transações simultâneas não se atrapalham
D	Durabilidade	Depois do `COMMIT`, os dados sobrevivem a falhas

Níveis de isolamento

Nível	Leitura suja	Leitura não repetível	Leitura fantasma
`READ UNCOMMITTED`	Possível	Possível	Possível
`READ COMMITTED`	Evita	Possível	Possível
`REPEATABLE READ` (**padrão do InnoDB**)	Evita	Evita	Evita na maioria dos casos (usa MVCC e *gap locks*)
`SERIALIZABLE`	Evita	Evita	Evita

```
SELECT @@transaction_isolation;
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

O InnoDB usa **MVCC**: leituras comuns enxergam um "retrato" consistente dos dados e não bloqueiam escritas.

Locks

```
START TRANSACTION;
SELECT saldo FROM contas WHERE id = 1 FOR UPDATE;   -- trava a linha até o fim da transação
UPDATE contas SET saldo = saldo - 100 WHERE id = 1;
COMMIT;
```

- `FOR UPDATE`: trava para escrita; outras transações esperam.
- `FOR SHARE`: permite leitura por outros, bloqueia escrita.
- `SKIP LOCKED` e `NOWAIT` (8.0+) evitam esperar, úteis em filas de tarefas.

Deadlock

Ocorre quando duas transações esperam uma pela outra. O InnoDB detecta, aborta uma delas (erro 1213) e a aplicação deve **tentar de novo**. Para reduzir a chance:

- Acesse tabelas e linhas sempre na mesma ordem.

- Mantenha transações curtas.
- Use índices adequados, para travar menos linhas.

Para investigar: `SHOW ENGINE INNODB STATUS;`.

> **Atenção:** comandos DDL (`CREATE`, `ALTER`, `DROP`, `TRUNCATE`) fazem *commit implícito* e não podem ser desfeitos.

9. Programação no Banco: Views, Procedures, Triggers e Events

Views

Uma view é uma consulta salva com nome. Simplifica o acesso e pode esconder colunas sensíveis. O MySQL **não tem** materialized view nativa; para isso, use uma tabela de resumo atualizada por `EVENT` ou pela aplicação.

```
CREATE OR REPLACE VIEW vw_total_por_cliente AS
SELECT c.id, c.nome, COALESCE(SUM(i.quantidade * pr.preco), 0) AS total
FROM clientes c
LEFT JOIN pedidos p      ON p.cliente_id = c.id AND p.status = 'pago'
LEFT JOIN itens_pedido i ON i.pedido_id = p.id
LEFT JOIN produtos pr    ON pr.id = i.produto_id
GROUP BY c.id, c.nome;

SELECT * FROM vw_total_por_cliente WHERE total > 500;
```

Stored procedures e functions

A diferença: a **function** devolve um valor e pode ser usada dentro de um `SELECT`; a **procedure** é chamada com `CALL` e pode ter parâmetros de saída. O `DELIMITER` troca o terminador temporariamente, para o cliente não cortar o corpo no primeiro `;`.

```
DELIMITER $$

CREATE PROCEDURE sp_fechar_pedido(IN p_pedido INT, OUT p_total DECIMAL(10,2))
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;

  START TRANSACTION;
```

```
    SELECT SUM(i.quantidade * pr.preco) INTO p_total
    FROM itens_pedido i JOIN produtos pr ON pr.id = i.produto_id
    WHERE i.pedido_id = p_pedido;

    UPDATE pedidos SET status = 'pago' WHERE id = p_pedido;
  COMMIT;
END$$

CREATE FUNCTION fn_desconto(p_preco DECIMAL(10,2), p_pct INT)
RETURNS DECIMAL(10,2) DETERMINISTIC
BEGIN
  RETURN p_preco * (1 - p_pct / 100);
END$$

DELIMITER ;

CALL sp_fechar_pedido(1, @total);
SELECT @total;
SELECT nome, fn_desconto(preco, 10) FROM produtos;
```

Estruturas disponíveis dentro do corpo: `DECLARE`, `SET`, `IF ... THEN ... ELSEIF ... END IF`, `CASE`, `WHILE`, `REPEAT`, `LOOP` e cursores (`DECLARE c CURSOR FOR ...`).

Triggers

Código executado automaticamente antes ou depois de `INSERT`, `UPDATE` ou `DELETE`. Dentro dele, `NEW` é a linha nova e `OLD` é a linha antiga.

```
CREATE TABLE auditoria_preco (
  produto_id INT, preco_antigo DECIMAL(10,2), preco_novo DECIMAL(10,2),
  alterado_em DATETIME DEFAULT CURRENT_TIMESTAMP
);

DELIMITER $$
CREATE TRIGGER trg_produtos_preco
AFTER UPDATE ON produtos
FOR EACH ROW
BEGIN
  IF OLD.preco <> NEW.preco THEN
    INSERT INTO auditoria_preco (produto_id, preco_antigo, preco_novo)
    VALUES (OLD.id, OLD.preco, NEW.preco);
  END IF;
END$$
DELIMITER ;
```

> **Use com moderação:** lógica escondida em triggers é difícil de depurar e pesa em cargas grandes. Auditoria e valores derivados são os usos mais seguros.

Events (agendador)

Rodam comandos em horários definidos, como um cron dentro do banco. Exigem `SET GLOBAL event_scheduler = ON;`.

```
CREATE EVENT ev_limpar_pedidos_abertos
ON SCHEDULE EVERY 1 DAY STARTS '2026-10-09 03:00:00'
DO
  DELETE FROM pedidos WHERE status = 'aberto' AND data < CURDATE() - INTERVAL
30 DAY;
```

Outros recursos úteis

Recurso	Para que serve
Colunas geradas	`total DECIMAL(10,2) AS (qtd * preco) STORED`, com possibilidade de indexar
Tabelas temporárias	`CREATE TEMPORARY TABLE`, existem só na sessão
Particionamento	`PARTITION BY RANGE (YEAR(data))`, para tabelas muito grandes
Variáveis de usuário	`SET @x = 10; SELECT @x;`
Prepared statements	`PREPARE`, `EXECUTE`, `DEALLOCATE PREPARE`

10. Segurança, Usuários e Backup

Usuários e permissões

O MySQL identifica o usuário pelo par `'nome'@'host'`. Siga o **princípio do menor privilégio**: a aplicação nunca deve conectar como `root`.

```
CREATE USER 'app_loja'@'localhost' IDENTIFIED BY 'SenhaForte#2026';

GRANT SELECT, INSERT, UPDATE, DELETE ON loja.* TO 'app_loja'@'localhost';
GRANT SELECT ON loja.vw_total_por_cliente TO 'relatorios'@'%';

SHOW GRANTS FOR 'app_loja'@'localhost';
REVOKE DELETE ON loja.* FROM 'app_loja'@'localhost';

ALTER USER 'app_loja'@'localhost' IDENTIFIED BY 'NovaSenha#2026';
DROP USER 'app_loja'@'localhost';
```

No MySQL 8 também há **roles** (`CREATE ROLE leitura; GRANT SELECT ON loja.* TO leitura; GRANT leitura TO 'ana'@'%';`), que facilitam gerenciar grupos de usuários.

SQL Injection

Acontece quando a entrada do usuário é concatenada dentro da consulta. Um valor como `' OR '1'='1` muda o sentido do comando.

```
// ERRADO: concatenação
$sql = "SELECT * FROM usuarios WHERE login = '$login'";

// CERTO: consulta parametrizada (PDO)
$stmt = $pdo->prepare('SELECT * FROM usuarios WHERE login = :login');
$stmt->execute(['login' => $login]);
```

```
# CERTO em Python (mysql-connector)
cursor.execute("SELECT * FROM usuarios WHERE login = %s", (login,))
```

> **Sempre use consultas parametrizadas** (prepared statements) ou um ORM configurado corretamente. Escapar texto manualmente não é suficiente. Nomes de tabelas e colunas não podem ser parâmetros: valide-os contra uma lista fixa.

Outras boas práticas de segurança

- Guarde senhas com hash e salt (bcrypt ou argon2), nunca em texto puro nem com `MD5`.
- Não exponha a porta 3306 à internet; restrinja por firewall ou VPN e use `bind-address`.
- Use SSL/TLS nas conexões remotas.
- Mantenha o servidor atualizado e remova usuários e bancos de teste (`mysql_secure_installation`).

- Dados sensíveis (CPF, cartão) exigem criptografia e cuidado com a LGPD; guarde o mínimo necessário.

Backup e restauração

```
# Backup lógico de um banco (consistente para InnoDB)
mysqldump -u root -p --single-transaction --routines --triggers --events loja
> loja.sql

# Todos os bancos, compactado
mysqldump -u root -p --all-databases --single-transaction | gzip >
backup_total.sql.gz

# Restaurar
mysql -u root -p loja < loja.sql

# Exportar e importar uma tabela em CSV
SELECT * INTO OUTFILE '/var/lib/mysql-files/clientes.csv'
  FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n'
FROM clientes;
LOAD DATA INFILE '/var/lib/mysql-files/clientes.csv' INTO TABLE clientes
  FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n';
```

Estratégia	Quando usar
`mysqldump`	Bancos pequenos e médios; simples e portátil
Percona XtraBackup / MySQL Enterprise Backup	Bancos grandes, backup físico a quente
Binary log (`binlog`)	Recuperação até um ponto no tempo (point-in-time)
Replicação	Alta disponibilidade e leitura em réplicas; **não substitui** backup

> **Regra 3-2-1:** três cópias, em dois tipos de mídia, com uma fora do local. E teste a restauração de tempos em tempos: backup que nunca foi restaurado não é backup confiável.

11. Erros Comuns, Exercícios e Plano de Estudo

Erros comuns no MySQL

Erro	Consequência	Como evitar
`UPDATE` / `DELETE` sem `WHERE`	Altera ou apaga a tabela inteira	Teste com `SELECT`; use transação
`= NULL` em vez de `IS NULL`	Nunca retorna linhas	Use `IS NULL` ou `<=>`
`NOT IN` com subconsulta que tem `NULL`	Resultado vazio	Use `NOT EXISTS`
`JOIN` sem condição	Produto cartesiano gigante	Sempre escreva o `ON`
Filtro da direita no `WHERE` de um `LEFT JOIN`	Vira `INNER JOIN` sem querer	Mova a condição para o `ON`
Contar duplicados por causa de um `JOIN` 1:N	Totais inflados	`COUNT(DISTINCT ...)` ou agregue antes de juntar
Função sobre coluna indexada no `WHERE`	O índice não é usado	Reescreva como intervalo
Usar `utf8` em vez de `utf8mb4`	Emojis e alguns caracteres quebram	Declare `utf8mb4`
Usar `FLOAT` para dinheiro	Erros de arredondamento	`DECIMAL(10,2)`
Confiar na ordem sem `ORDER BY`	Ordem imprevisível	Declare a ordenação
Concatenar com `||`	No MySQL isso é `OR`	Use `CONCAT()`
Montar SQL por concatenação de texto	SQL injection	Consultas parametrizadas

Exercícios

Use a base da seção 2. Tente resolver antes de ver a resolução na documentação ou em um colega.

#	Nível	Enunciado
1	Básico	Liste nome e cidade dos clientes de Niterói em ordem alfabética
2	Básico	Mostre os 5 produtos mais caros
3	Básico	Quantos clientes têm cidade cadastrada?
4	Intermediário	Por categoria, mostre quantidade de produtos e preço médio, só onde houver mais de 3 produtos
5	Intermediário	Liste nome do cliente e data de cada pedido feito em 2026
6	Intermediário	Liste os clientes que nunca fizeram pedido
7	Intermediário	Mostre os produtos com preço acima da média da própria categoria
8	Avançado	Calcule o valor total de cada pedido e o ranking dos pedidos por valor em cada mês
9	Avançado	Para cada cliente, mostre o valor de cada pedido e a diferença para o pedido anterior (`LAG`)
10	Avançado	Retorne os 3 produtos mais vendidos de cada categoria
11	Avançado	Calcule o total de vendas acumulado mês a mês no ano
12	Projeto	Crie uma procedure que registre um pedido com vários itens em uma única transação e desfaça tudo em caso de erro

Plano de estudo sugerido

Etapa	Foco	Prática
1	Seções 1 a 3: instalar, criar tabelas, `SELECT`	Exercícios 1 a 3
2	Seções 4 e 5: agregações, `JOIN` , subconsultas	Exercícios 4 a 7
3	Seção 6: CTEs e window functions	Exercícios 8 a 11

Etapa	Foco	Prática
4	Seções 7 e 8: índices, `EXPLAIN`, transações	Medir uma consulta antes e depois de criar um índice
5	Seção 9: views, procedures, triggers	Exercício 12
6	Seção 10: usuários, backup	Fazer e restaurar um `mysqldump`
7	Modelagem	Modelar do zero um sistema (biblioteca, clínica, delivery)

Onde praticar: SQLBolt, SQLZoo, HackerRank (SQL), LeetCode (Database), Mode SQL Tutorial. **Para aprofundar:** documentação oficial do MySQL 8 (dev.mysql.com/doc), o livro *High Performance MySQL* e o site *Use the Index, Luke!* (use-the-index-luke.com), gratuito, sobre índices.

Cheat sheet

```
SELECT   col, AGG(col)              -- o que mostrar
FROM     tabela a
JOIN     outra b ON a.id = b.a_id   -- combinar tabelas
WHERE    condição_de_linha          -- filtrar linhas
GROUP BY col                        -- agrupar
HAVING   condição_de_grupo          -- filtrar grupos
ORDER BY col DESC                   -- ordenar
LIMIT    10 OFFSET 0;               -- paginar
```

Preciso de...	Use
Concatenar texto	`CONCAT()`
Valor padrão para `NULL`	`IFNULL()` / `COALESCE()`
Data atual	`NOW()` / `CURDATE()`
Formatar data	`DATE_FORMAT(d, '%d/%m/%Y')`
Juntar valores de um grupo	`GROUP_CONCAT()`
Inserir ou atualizar	`INSERT ... ON DUPLICATE KEY UPDATE`
Último id inserido	`LAST_INSERT_ID()`
Ranking por grupo	`ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)`
Ver o plano da consulta	`EXPLAIN` / `EXPLAIN ANALYZE`

Preciso de...	Use
Backup	`mysqldump --single-transaction`

> **Lembretes finais:** `NULL` só se compara com `IS NULL` · `WHERE` filtra antes de agrupar, `HAVING` depois · `utf8mb4` sempre · `InnoDB` sempre · parametrize toda consulta · meça com `EXPLAIN` antes de otimizar · teste seus backups.